# Ders 21 Genişletilmiş Notu: En Uzun Ortak Alt Dizinin Geri İzlenmesi, Graflara Giriş, DFS ve BFS

## Ders 21 Genişletilmiş Notu: En Uzun Ortak Alt Dizinin Geri İzlenmesi, Graflara Giriş, DFS ve BFS

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

## İçindekiler


- 1. En Uzun Ortak Alt Dizinin Geri İzlenmesi
- 2. Graflara Giriş ve Temel Kavramlar
- 3. Derinlik Öncelikli Arama (DFS)
- 4. Genişlik Öncelikli Arama (BFS)
- 5. DFS ve BFS Karşılaştırması ve Karmaşıklık
- 6. DFS ve BFS ile Çözülen Problemler
- 7. Topolojik Sıralamaya Giriş
- Örnek Sorular


## 1. En Uzun Ortak Alt Dizinin Geri İzlenmesi

Ders, dönemin 11. haftasındaydı. Hoca, geçen hafta dinamik programlama başlığında düzenleme uzaklığı (edit distance) ve en uzun ortak alt dizi (longest common subsequence, LCS) algoritmalarını gördüklerini hatırlattı. LCS'de iki dizinin ortak olan en uzun örüntüsü (pattern) aranır. Bu örüntünün karakterlerinin arka arkaya gelmesi gerekmez. Araya başka harfler girmiş olabilir. Bu harfler yanlışlıkla eklenmiş gibi düşünülür ve algoritma onlardan etkilenmeden en uzun örüntüyü bulur. Hoca, bu yaklaşımın DNA zincirleri gibi dizilerin karşılaştırılmasında çok sık kullanıldığını yeniden andı.

Geçen hafta dinamik programlama mantığıyla bir $C$ matrisi doldurulmuş ve LCS'nin uzunluğu bulunmuştu. Örnekte bu uzunluk 3'tü. Açık kalan soru şuydu: Bu 3 uzunluktaki örüntü hangi harflerden oluşuyor? Geçen hafta geriye doğru gidilerek bulunabileceği, ama bazı yerlerde birden fazla yol olabileceği konuşulmuş ve konu orada bırakılmıştı. Bu derste o eksik kısım için ek kod yazıldı.

### 1.1 Üç İşlem: Ekle, X'te Atla, Y'de Atla

Hoca, geçen hafta "skip x" ve "skip y" kavramlarından söz ettiklerini hatırlattı. Temelde üç iş vardır:

- **Ekle (add):** İlgili karakter LCS'ye alınır. Bu, $x_i = y_j$ olduğu durumdur.
- **X'te atla (skip x):** $x$ dizisinde ileri gidilir, $x_i$ karakteri alınmaz.
- **Y'de atla (skip y):** $y$ dizisinde ileri gidilir, $y_j$ karakteri alınmaz.

Hangi işlemin yapıldığını saklamak için $C$ matrisinin yanında ikinci bir matris tutulur. Hoca buna $B$ dedi. $C$ matrisi doldurulurken her hücre için $B$'ye de bir işaret yazılır. Hoca, rahat anlaşılsın diye işaretleri "add", "skip x", "skip y" diye yazdı. Matrisin içine bunlar yerine 0, 1 ve 2 gibi sayılar konabilir.

Eşitlik yoksa hücrenin değeri iki yerden gelebilir: üst satırdan ($C[i-1,j]$) ya da soldaki sütundan ($C[i,j-1]$). Bunların büyüğü alınır, çünkü daha uzun ortak örüntü bizim için avantajlıdır. Hangisinin alındığı da $B$'ye yazılır:

```
if x[i] == y[j]:
    C[i,j] <- C[i-1,j-1] + 1
    B[i,j] <- add              # örneğin 0
else if C[i-1,j] >= C[i,j-1]:
    C[i,j] <- C[i-1,j]
    B[i,j] <- skip x           # örneğin 1: x[i] LCS'de yok
else:
    C[i,j] <- C[i,j-1]
    B[i,j] <- skip y           # örneğin 2: y[j] LCS'de yok
```

$C[i-1,j] \geq C[i,j-1]$ ise değer üst satırdan gelmiştir. Bu, $x$ dizisi üzerinde yola devam edildiği, yani $x_i$'nin atlandığı anlamına gelir. Aksi hâlde değer soldaki sütundan gelmiştir. Bu durumda $y_j$ atlanmıştır ve $y_j$ LCS'de yer almaz.

> **Ders sonrası not:** Hoca bir yerde eşitlik olmayan durum için "üst satırdan ya da sol üstünden bilgi gelebilir, en büyüğünü alt çapraza taşırız" dedi. Doğrusu: Eşitlik yoksa değer üstteki hücreden ($C[i-1,j]$) ya da soldaki hücreden ($C[i,j-1]$) gelir. Sol üst çapraz ($C[i-1,j-1]$) yalnız eşitlik durumunda kullanılır.

### 1.2 Geri İzleme Kodu

$B$ matrisi oluştuktan sonra örüntünün hangi harflerden oluştuğu bu matrisle bulunur. Hocanın yazdığı kod parçası:

```
i <- m ; j <- n
while i != 0 and j != 0:
    switch B[i,j]:
        case add:
            LCS'nin en önüne x[i] ekle
            i <- i - 1 ; j <- j - 1
            break
        case skip x:
            i <- i - 1
            break
        case skip y:
            j <- j - 1
            break
```

Kodun her durumu bir yöne karşılık gelir:

- **add:** Bu, eşleşen son karakterdir. Bu yüzden LCS'nin en önüne eklenir. Çaprazdan gelindiği için hem $i$ hem $j$ bir azaltılır.
- **skip x (1):** Değer yukarıdan gelmiştir. Bir sonraki adımı bulmak için bir üst satıra çıkılır.
- **skip y (2):** Değer soldan gelmiştir. $j$ bir azaltılır, bir sol sütuna geçilir.

Hoca, geçen dersteki "BCB mi, BDB mi, ikisi de olabilir" tartışmasının cevabının bu yapı olduğunu söyledi. Örüntüyü bulmak için ikinci bir matrise ve ek bir koda ihtiyaç vardır.

> **Hocanın vurgusu:** LCS'nin yalnız uzunluğunu bulmak yetmez. Örüntünün hangi karakterlerden oluştuğunu bulmak için $B$ matrisi ve geri izleme kodu gerekir.

### 1.3 Geri İzleme Nereden Başlar?

Hoca öğrencilere önce birinci kısımda ne yapıldığını, sonra geri izlemenin nereden başladığını sordu.

**Soru:** Birinci kısımda ne yapıyoruz? $B$ ve $C$ matrisleri hangi bilgiyi tutuyor? **Cevap:** $C$ matrisi en uzun ortak örüntünün kaç uzunlukta olduğunu tutar. $B$ matrisi ise her hücrede hangi işlemin yapıldığını tutar: ekleme mi, x'te atlama mı, y'de atlama mı.

**Soru:** Örüntüyü bulmak için $B[i, j]$'ye nereden itibaren bakacağız? **Cevap:** Bir öğrenci, matrisin en son doldurulan yerinden başa doğru bakılacağını söyledi. Hoca, bunun koddan da okunabildiğini ekledi: Döngü "$i \neq 0$ ve $j \neq 0$ olduğu sürece" çalışıyor. Demek ki sondan, yani sağ alt köşeden ($i = m$, $j = n$) başlanır.

### 1.4 Örnek: BACDB ve BDCB

Hocanın andığı örnek, ders 19'daki $x$ = BACDB (satırlar) ve $y$ = BDCB (sütunlar) dizileridir. Derste bu örnek yeniden izlenmedi. Aşağıdaki izleme, hocanın yazdığı kurallarla Python'da yapıldı (ara adım eklendi).

$C$ matrisi (ilk satır ve sütun 0):

| | | B | D | C | B |
|---|---|---|---|---|---|
| | 0 | 0 | 0 | 0 | 0 |
| B | 0 | 1 | 1 | 1 | 1 |
| A | 0 | 1 | 1 | 1 | 1 |
| C | 0 | 1 | 1 | 2 | 2 |
| D | 0 | 1 | 2 | 2 | 2 |
| B | 0 | 1 | 2 | 2 | 3 |

$B$ matrisi (0 = add, 1 = skip x, 2 = skip y; eşitlikte $\geq$ kuralıyla skip x):

| | B | D | C | B |
|---|---|---|---|---|
| B | 0 | 2 | 2 | 0 |
| A | 1 | 1 | 1 | 1 |
| C | 1 | 1 | 0 | 2 |
| D | 1 | 0 | 1 | 1 |
| B | 0 | 1 | 1 | 0 |

Geri izleme $(i, j) = (5, 4)$'ten başlar:

- $(5, 4)$: add. B eklenir. LCS = "B". $(4, 3)$'e geçilir.
- $(4, 3)$: skip x. $(3, 3)$'e geçilir.
- $(3, 3)$: add. C eklenir. LCS = "CB". $(2, 2)$'ye geçilir.
- $(2, 2)$: skip x. $(1, 2)$'ye geçilir.
- $(1, 2)$: skip y. $(1, 1)$'e geçilir.
- $(1, 1)$: add. B eklenir. LCS = "BCB". $(0, 0)$'a gelinir, döngü biter.

Sonuç BCB'dir. Eşitlik durumunda skip y seçilseydi (yani koşul $>$ olsaydı) aynı izleme BDB'yi verirdi. İki sonuç da 3 uzunluktadır. Hangisinin bulunacağı, eşitlikte hangi yönün seçildiğine bağlıdır.

## 2. Graflara Giriş ve Temel Kavramlar

Dinamik programlama burada tamamlandı ve graflara geçildi. Hoca, öğrencilerin veri yapıları dersinde graflarla ilgili temel bilgileri aldığını, burada bazı şeylerin hatırlatılacağını, bazılarının da yeni ekleneceğini söyledi.

### 2.1 Graflarla Çözülen Problemler

Hoca önce graf yaklaşımıyla hangi problemlerin çözülebileceğini sordu.

**Soru:** Graflarla hangi problemleri çözebiliriz? **Cevap:** Bir öğrenci en kısa yol problemini söyledi. Hoca bunu onayladı. En kısa yol için kullanılan algoritma olarak Dijkstra algoritması anıldı. Öğrenci genişlik öncelikli arama tarzında yaklaşımları da saydı. Bir başka öğrenci ikili arama ağacını (binary search tree) örnek verdi ve mantığını anlattı: Düğümlerden oluşur, her düğümün en fazla iki çocuğu olur ve değerler küçükten büyüğe bir düzenle yerleşir.

Hoca buradan şu sonuca vardı: Ağaç (tree), grafın özel bir hâlidir. Hocanın saydığı diğer kullanım alanları şunlardır:

- **Torrent:** Bir dosya indirilirken hangi parçanın hangi düğümden geleceği graf yapısıyla belirlenir.
- **Web tarama (crawling):** Web sayfalarındaki ve onların altındaki sayfalardaki bilgiler çekilirken graf yapısından yararlanılır.
- **Topolojik sıralama (topological sort):** Bağımlılıklar (dependency) olan problemlerde kullanılır. Örnekler: derslerin ön koşulları, bir yazılımı kurmadan önce kurulması gereken yazılımlar ve onların da ön koşulları. Bu problem dersin sonunda ele alındı.
- **Sosyal ağlar (social network):** Graf yapısı burada yoğun olarak kullanılır.

### 2.2 Ağaç Olma Koşulu

**Soru:** Bir grafın ağaç olma koşulu nedir? Graftan ağaç oluşturup oluşturamayacağınızı nasıl anlarsınız? **Cevap:** Bir öğrenci döngü (cycle) oluşmaması gerektiğini söyledi. Hoca literatürdeki adını sordu. Bir öğrenci "cyclic" dedi, hoca da onayladı. Hocaya göre graf döngü içeriyorsa, yani döngülü (cyclic) bir yapıya sahipse, o graftan bir ağaç oluşturma şansı yoktur.

> **Ders sonrası not:** Aranan özelliğin adı döngülülük değil, döngüsüzlüktür (acyclicity). Ayrıca yalnız döngüsüz olmak yetmez: Bir graf, bağlı (connected) ve döngüsüzse ağaçtır. Döngüsüz ama bağlı olmayan graf ormandır (bkz. 2.7). $n$ düğümlü bir ağaçta $n - 1$ kenar vardır. Döngülü bir grafın kendisi ağaç değildir. Ama bağlıysa, bazı kenarları atılarak ondan bir kapsayan ağaç (spanning tree) elde edilebilir.

### 2.3 Düğüm, Kenar, Yönlü ve Yönsüz Graf

Hoca graftaki temel bilgileri hatırlattı. Grafta düğüm (vertex) ve kenar (edge) vardır. Grafların farklı çeşitleri de vardır:

- **Yönlü graf (directed graph, digraph):** Kenarların bir yönü vardır.
- **Yönsüz graf (undirected graph):** Kenarların yönü yoktur.

### 2.4 Grafları Saklama: Komşuluk Matrisi ve Komşuluk Listesi

**Soru:** Graflardaki düğüm ve kenar bilgilerini hangi iki yapıda saklıyoruz? **Cevap:** Bir öğrenci komşuluk matrisi (adjacency matrix) ve komşuluk listesini (adjacency linked list) söyledi. Hoca onayladı.

**Soru:** Hangisini kullanacağımızı neye göre belirliyoruz? **Cevap:** Bir öğrenci düğüm sayısını önerdi, çünkü graf büyüdükçe matris seyrekleşir. Hoca, yalnız düğüm sayısı denemeyeceğini söyledi. Öğrenci bunun üzerine bağlantı sayısını söyledi. Hocaya göre burada kenar sayısı önem kazanır, ama asıl belirleyici olan kenar ve düğüm sayısının birlikte durumudur.

Hoca burada iki kavramı anlattı: yoğun (dense) ve seyrek (sparse). Kenar sayısı yüksekse graf yoğundur ve matris yapısıyla saklanır. Kenar sayısı azsa tercih komşuluk listesinden yana olur. Hocanın örneği: 5 düğümlü bir grafı $5 \times 5$ bir matrisle sakladığınızı düşünün. Bu beş düğüm arasında yalnız iki kenar varsa matrisin geri kalanı sıfırdır. Yani büyük bir matrisin çoğu boşa yer kaplar.

> **Ders sonrası not:** Hoca yoğun grafı "düğümlere yakın sayıda kenar varsa" diye tanımladı. Doğrusu: Kenar sayısı en fazla olabilecek kenar sayısına, yani $|V|^2$ mertebesine yakınsa graf yoğundur. Kenar sayısı $|V|$ mertebesindeyse graf seyrektir ve komşuluk listesi daha uygundur.

### 2.5 Tam Graf ve Ağırlıklı Graf

**Soru:** Bir grafın tam graf (complete graph) olup olmadığını nasıl anlarız? **Cevap:** Bir öğrenci, bütün düğümler birbirine bağlı olunca grafın tam olduğunu söyledi. Hoca bunu onayladı ve şöyle açıkladı: Herhangi bir düğüme bir kenar gitmiyorsa o graf tam değildir. Tüm düğümlerin mutlaka bir kenar bağlantısı varsa graf tam graf olarak adlandırılır.

> **Ders sonrası not:** Hocanın açıklaması, her düğümün en az bir kenarı olmasını, yani yalıtılmış düğüm bulunmamasını anlatıyor. Bu, tam graf için yetmez. Tam grafta her düğüm çifti doğrudan bir kenarla bağlıdır. Öğrencinin "tüm düğümler birbirine bağlı" tanımı doğrudur. $n$ düğümlü yönsüz tam grafta $n(n-1)/2$ kenar vardır.

**Soru:** Ağırlıklı graf (weighted graph) deyince ne anlıyorsunuz? İki düğüm arasında bağlantı yoksa ne yapıyoruz? **Cevap:** Bir öğrenci "ağırlıklı graf, her düğümün belli bir ağırlığı var" dedi. Hoca, bağlantı olmayan durumu sordu. Örneğin A ile B arası 5 olarak tanımlanmış, ama A ile C arasında bağlantı yok. Bu durumda değer sonsuz ($\infty$) olarak tanımlanır. Hoca, bazen eksi bir değerin de kullanılabildiğini, ama kenarlarda eksi ağırlıklar kullanmak istiyorsanız sonsuzu, yani çok büyük bir sayıyı tercih ettiğinizi söyledi.

Ağırlıklı graf matriste bu şekilde saklanır. Komşuluk listesinde ise her komşu için iki bilgi tutulur: hangi düğüme bağlı olduğu ve hangi ağırlıkla bağlı olduğu.

> **Ders sonrası not:** Ağırlıklar düğümlere değil kenarlara aittir. Öğrencinin "her düğümün ağırlığı var" ifadesi derste düzeltilmedi. Ağırlıklı grafta her kenarın bir ağırlığı (maliyet, mesafe gibi) vardır.

### 2.6 Döngüsüzlük, Bağlılık ve Bağlı Bileşenler

Hoca iki özelliği daha andı: döngüsüzlük (acyclicity) ve bağlılık (connectivity).

Döngüsüzlük için hoca tahtaya A, B, C, D, E düğümlerinden oluşan döngüsüz bir graf çizdi. Bu grafa bir kenar eklendiğinde B, D, E arasında bir döngü oluşuyordu. Artık graf döngüsüz değildir. Hoca, bazı problemlerde grafın döngüsüz olup olmadığını kontrol etmenin gerekeceğini, bunu arayan birçok problemle karşılaşılacağını söyledi.

**Soru:** Bağlılık özelliğini nasıl kontrol ediyoruz? **Cevap:** Bir öğrenci, her düğüm çifti için bir düğümden diğerine ulaşan bir yol olması gerektiğini söyledi. Hoca bunu "gayet güzel bir tarif" diye onayladı.

Hoca grafa F ve G düğümlerini ekledi. F ile G birbirine bağlıydı, ama A, B, C, D, E kümesine bağlı değildi. Bu durumda her düğüme diğer düğümler üzerinden erişilemez. Örneğin G'den A'ya gidilemez. Bu grafta iki bağlı bileşen (connected component) vardır: ABCDE ve FG. Bir grafın bağlı olması için bir düğümden diğer tüm düğümlere ulaşılabilmelidir. Böyle değilse graf birden fazla bağlı bileşenden oluşur.

### 2.7 Orman ve Rastgele Orman

Hoca ardından birbirine bağlı olmayan iki ağaçtan oluşan bir yapı çizdi ve bunun adını sordu.

**Soru:** Birden fazla ağaçtan oluşan yapıya ne ad verilir? **Cevap:** Orman (forest).

Hoca konuya bir dipnot olarak rastgele orman (random forest) algoritmasını ekledi. Bir öğrenci bunun bir sınıflandırma algoritması olduğunu söyledi. Hoca şöyle anlattı: Karar ağacı (decision tree) algoritmalarında tek bir ağaç vardır ve her düğümde bir karar verilir. Genelde düğümlerde özelliklere bakılır. Örneğin "boyu 1,70'ten uzun olanlar ve olmayanlar", boyu 1,70'ten uzunsa "kilosu şundan az olanlar ve fazla olanlar" gibi, her düğümde bir özellik karşılaştırılır. Rastgele orman ise karar ağaçlarından yararlanan, kollektif öğrenme yaklaşımını benimseyen bir algoritmadır. Birden fazla ağaç vardır ve bu ağaçlar birbirinden bağımsız olarak karar verir. Sonunda her ağacın kararı ortak bir noktada toplanır ve bir uzlaşıyla (konsensüs) sınıflandırma kararı verilir.

## 3. Derinlik Öncelikli Arama (DFS)

Hoca, graflarda en çok duyulan iki algoritmanın derinlik öncelikli arama (depth-first search, DFS) ve genişlik öncelikli arama (breadth-first search, BFS) olduğunu söyledi. Bu iki algoritma geçen dönem (veri yapıları dersinde) anlatılmıştı.

**Soru:** Hiçbir şeye bakmadan bu iki algoritmanın nasıl çalıştığını söyleyebilecek olan var mı? **Cevap:** Bir öğrenci şöyle açıkladı: Derinlik öncelikli aramada kaynak düğümden önce gidebildiğimiz kadar uzağa gideriz, sonra diğer yollara geçeriz. Genişlik öncelikli aramada ise önce

yakındaki bütün düğümleri gezeriz, sonra gezdiğimiz düğümlerin en yakınlarını gezeriz ve bu şekilde dağılarak ilerleriz. Hoca bunu "gayet güzel bir açıklama" diye onayladı.

**Soru:** DFS ve BFS hangi veri yapılarını kullanır? **Cevap:** DFS yığın (stack), BFS kuyruk (queue) kullanır.

Hoca, iki algoritmanın farklı problemler için kullanıldığını, ama aynı problemler için de kullanılabildiğini söyledi. Sonuçta ikisi de graf gezme algoritmasıdır ve ikisinde de tüm düğümler gezilir.

### 3.1 Özyineleme ve Durma Noktası

**Soru:** DFS'i gerçekleştirirken hangi temel algoritmik yaklaşımı kullanıyoruz? **Cevap:** Özyinelemeli (recursive) yaklaşım.

**Soru:** Özyinelemede durma noktası neresidir? **Cevap:** Bir öğrenci "tüm düğümlerin ziyaret edildiği durum" dedi. Hoca bunun bütün algoritmanın sonlanma koşulu olduğunu, özyinelemenin sonlanmasının ise başka bir şey olduğunu belirtti. Bir başka öğrenci doğru cevabı verdi: Bir düğümden gidilecek başka bir düğüm bulunamadığında, yani ziyaret edilmemiş bir komşu kalmadığında özyineleme biter. Hoca, bunun yığında herhangi bir bilgi kalmayana kadar devam etmek anlamına geldiğini ekledi.

### 3.2 Sözde Kod

Hoca hatırlatma amaçlı sözde kodu (pseudocode) yazdı. Önce graftaki tüm düğümler 0 ile, yani "ziyaret edilmedi" olarak işaretlenir. Bir sayaç (count) 0'a eşitlenir. Sonra bir `for` döngüsüyle graftaki düğümler gezilir. İşareti 0 olan, yani henüz ziyaret edilmemiş her düğüm için özyinelemeli fonksiyon çağrılır.

```
DFS(G):
    mark each vertex in V with 0      # ziyaret edilmedi
    count <- 0
    for each vertex v in V do
        if v is marked with 0
            dfs(v)

dfs(v):
    count <- count + 1
    mark v with count
    for each vertex w in V adjacent to v do
        if w is marked with 0
            dfs(w)
```

Özyinelemeli fonksiyonda sayaç bir artırılır ve düğüm bu sayaçla işaretlenir. Sonra düğümün komşularına bakılır. Bir komşu daha önce ziyaret edilmediyse, o komşunun komşularına ulaşmak için fonksiyon kendisini yeniden çağırır. Hoca, mantığın çok basit olduğunu söyledi: Ziyaret edilen düğüm sırayla yığına eklenir (push).

**Soru:** Yığından çıkarma (pop) işlemini ne zaman yapıyoruz? **Cevap:** Devam edilebilecek bir kenar, gidilecek başka bir düğüm kalmadığında. Hoca şöyle açıkladı: Ulaşılan düğümün etrafında ziyaret edilmemiş başka bir düğüm kalmadıysa pop yapılır ve bir önceki düğüme dönülür. Onun komşularına bakılır. Ziyaret edilmemiş komşu varsa derinlemesine gitmeye devam edilir. Aşağı inildikten sonra yine ziyaret edilmemiş düğüm kalmadıysa yine pop yapılır ve yığın sırayla boşaltılır.

### 3.3 Örnek: A'dan G'ye Graf

Hoca, bağlılık konusunda çizdiği grafı kullandı. Altyazıdan çıkan kenarlar şunlardır: A-B, B-C, B-D, B-E, D-E ve F-G.

Hoca, genelde alfabetik sırayla başlandığını, eşitlik olduğunda da alfabetik sıranın tercih edildiğini söyledi. İzleme şöyledir:

- A'dan başlanır. A'dan B'ye gidilir.

- B'den gidilebilecek üç seçenek vardır: C, D, E. Alfabetik sırayla C seçilir.
- C'nin gidecek yeri kalmamıştır. Bağlı olduğu tek düğüm olan B zaten ziyaret edilmiştir. C yığından çıkarılır, B'ye dönülür.
- B'den D'ye, D'den E'ye gidilir.
- E, B'ye ve D'ye bağlıdır. İkisi de ziyaret edilmiştir. E çıkarılır.
- Sonra sırayla D, B ve A çıkarılır. Yığın boşalır.

Python ile doğrulanan sıralar:

| Sıra | Düğümler |
|---|---|
| Ziyaret (push) sırası | A, B, C, D, E, F, G |
| Çıkarma (pop) sırası | C, E, D, B, A, G, F |

Yığın boşaldığında ilk bağlı bileşen gezilmiş olur. Hoca burada sözde kodun neresinde olunduğunu sordu.

**Soru:** Bir bağlı bileşeni gezdik ve yığın boşaldı. Ama grafın tamamını henüz gezmedik. Bunu nereden anlıyoruz, sözde kodun neresindeyiz? **Cevap:** Bir öğrenci "if v is marked with 0" kısmında olunduğunu söyledi. Yani dıştaki "for each vertex v in V" döngüsüne dönülmüştür. Bu döngü ziyaret edilmemiş F'yi bulur ve yeni bir $dfs(F)$ çağrısı başlatır. F'den G'ye gidilir, sonra G ve F çıkarılır.

### 3.4 Bağlı Bileşen Sayısını Bulma

**Soru:** Bu algoritmayla bir graftaki bağlı bileşen sayısını bulabilir miyiz? Bulabiliriz ise sayma işini nerede yaparız? **Cevap:** Evet. Sayma dıştaki döngüde yapılır. DFS her çağrılıp döndüğünde bir bağlı bileşen bitmiş olur. Bu yüzden sayaç, dıştaki döngüde $dfs(v)$ çağrısının yanına konur. Örnekteki grafta $dfs$ iki kez çağrılır: A'dan ve F'den. Bağlı bileşen sayısı 2'dir.

## 4. Genişlik Öncelikli Arama (BFS)

Aradan sonra hoca, DFS'in yığın, push ve pop işlemlerini geçen dönem gördükleri için detaylı yapmadığını söyledi.

> **Hocanın vurgusu:** DFS ve BFS'in işleyişi sonradan sınavda karşınıza çıkabilir. Buna mutlaka hazırlıklı olun. Hatırlamadığınız bir yer varsa uygulama saatinde üzerinden geçilebilir.

### 4.1 Özyinelemeli mi, Yinelemeli mi?

BFS'te kuyruk kullanıldığı hatırlatıldıktan sonra hoca şu soruyu sordu.

**Soru:** DFS'te özyinelemeli yaklaşım kullanıyorduk. BFS'te de özyinelemeli yaklaşım mı kullanıyoruz, yoksa yinelemeli (iterative) yaklaşım mı? **Cevap:** Bir öğrenci yinelemeli yaklaşım kullanıldığını söyledi. Hoca "emin misin?" diye sordu. Hocaya göre BFS'te bir düğümden başlanıp ona komşu olan tüm düğümler gezilir, sonra bu düğümlerin kendi komşularına erişilir. Bu yüzden yinelemeli bir yaklaşım kurulamaz demediğini, ama BFS'e en uygun yaklaşımın özyinelemeli yaklaşım olduğunu söyledi.

> **Ders sonrası not:** Hoca "BFS'e en uygun yaklaşım özyinelemelidir" dedi. Doğrusu: Öğrencinin cevabı doğrudur. Hocanın hemen ardından yazdığı $bfs(v)$ fonksiyonu kendisini hiç çağırmaz. Kuyruk ve `while` döngüsüyle çalışır. Yani BFS yinelemelidir. Sözde kodda yalnız dıştaki `for` döngüsünün $bfs(v)$'yi çağırması vardır, bu bir özyineleme değildir. Özyineleme, yığın yapısıyla örtüştüğü için DFS'in doğasına uygundur.

### 4.2 Sözde Kod

Başlangıç DFS ile aynıdır. Graftaki her düğüm 0 ile işaretlenir, sayaç sıfırlanır ve ziyaret edilmemiş her düğüm için $bfs$ fonksiyonu çağrılır.

```
BFS(G):
    mark each vertex in V with 0
    count <- 0
    for each vertex v in V do
        if v is marked with 0
            bfs(v)

bfs(v):
    count <- count + 1
    mark v with count
    initialize a queue with v
    while the queue is not empty do
        for each vertex w in V adjacent to the front vertex do
            if w is marked with 0
                count <- count + 1
                mark w with count
                add w to the queue
        remove the front vertex from the queue
```

$bfs$ fonksiyonu yine sayacı bir artırarak başlar ve ilgili düğümü sayaçla işaretler. Sonra bu düğümle bir kuyruk oluşturulur. Kuyruk boşalmadığı sürece döngü devam eder. Döngünün içinde kuyruğun önündeki düğümün her komşusu için, daha önce ziyaret edilip edilmediği kontrol edilir. Ziyaret edilmemişse sayaç bir artırılır, komşu düğüm sayaçla işaretlenir ve kuyruğa eklenir. `for` döngüsü tamamlanınca en ön sıradaki düğüm kuyruktan çıkarılır.

### 4.3 Örnek

Hoca küçük bir örnek graf çizdi ve BFS ile hangi sırayla ilerleneceğini sordu. Altyazıdan çıkan kenarlar şunlardır: A'nın komşuları B, C, D; B'nin komşuları A, D, E. Ayrıca bir F düğümü vardı.

Bir öğrenci A'dan başlayıp B'ye geçti, ama B'den sonra hemen D ve E'ye bakmayı önerdi. Hoca bunun yanlış olduğunu söyledi: BFS'te kademe kademe ilerlenir. Önce A'nın bütün komşuları ele alınmalıdır. Bir başka öğrenci doğru sırayı verdi: A'dan B'ye, sonra C'ye, sonra D'ye. A'nın tek uzaklıkta gidilebilecek komşusu kalmayınca B'ye geçilir ve B'den E'ye gidilir. D zaten ziyaret edildiği için B'nin başka bir komşusu kalmaz. Sonra C'ye, D'ye bakılır ve F'ye ulaşılır.

Hoca bu anlatımın sözde koda nasıl karşılık geldiğini sordu ve birlikte izlediler:

- $bfs(A)$ çağrılır. A sayaçla işaretlenir ve A ile bir kuyruk oluşturulur. Kuyruk: [A].
- Öndeki düğüm A'dır. A'nın ziyaret edilmemiş komşuları B, C, D sırayla işaretlenip kuyruğa eklenir. Kuyruk: [A, B, C, D].
- A'nın 0 ile işaretli komşusu kalmamıştır. `for` döngüsü biter ve A kuyruktan çıkarılır. Kuyruk: [B, C, D].
- Öndeki düğüm artık B'dir. D ziyaret edildiği için yalnız E arkaya eklenir. Kuyruk: [B, C, D, E].
- B'nin başka komşusu kalmaz, B çıkarılır. Sonra C'ye geçilir.

Hoca izlemeyi burada bıraktı ve genel mantığın bu olduğunu söyledi. Python ile doğrulanan BFS sırası: A, B, C, D, E, F.

**Soru:** A'yı, sonra B, C, D'yi kuyruğa koyduk. Öndeki düğümü kuyruktan ne zaman çıkaracağım?
**Cevap:** A'nın gezilmemiş başka bir komşusu kalmadığı zaman. Sonra sırada B vardır.

## 5. DFS ve BFS Karşılaştırması ve Karmaşıklık

Hoca iki algoritmayı yan yana karşılaştırmak için bir tablo kurdu.

### 5.1 Veri Yapısı, Sıralama Sayısı ve Kenar Türleri

Veri yapısı tarafında DFS yığın, BFS kuyruk kullanır. Ardından düğüm sıralaması sayısı (number of vertex orderings) soruldu. Bu, düğümlere erişim sırasını ve düğümlerden ayrılma sırasını ifade eder.

**Soru:** DFS'te ve BFS'te kaç tane düğüm sıralaması vardır? **Cevap:** Bir öğrenci şöyle cevap verdi: BFS'te bir tanedir, çünkü düğümler kuyruğa konur ve aynı sırayla çıkarılır. DFS'te iki tanedir, çünkü düğümler hem yığına konur hem de arada çıkarılır ve bu iki sıra farklıdır. Hoca onayladı.

Kenar türleri tarafında hoca, DFS'te ağaç ve geri kenarlar (tree and back edges), BFS'te ağaç ve çapraz kenarlar (tree and cross edges) oluştuğunu söyledi ve bunların geçen sene anlatıldığını belirtti.

> **Ek bilgi (derste anlatılmadı):** Ağaç kenarı, gezinti sırasında yeni bir düğüme ulaşmak için kullanılan kenardır. Yönsüz bir grafta DFS sırasında daha önce ziyaret edilmiş bir ataya (ebeveyn dışında) giden kenar geri kenardır. BFS'te ise ziyaret edilmiş ve ata olmayan bir düğüme giden kenar çapraz kenardır. Bu yüzden yönsüz grafta geri ya da çapraz kenar görülmesi, grafta döngü olduğunu gösterir.

### 5.2 Karmaşıklığı Belirleyen Parametreler

**Soru:** Bu iki algoritmanın karmaşıklığını belirleyen parametreler nelerdir? **Cevap:** Bir öğrenci düğüm ve kenar sayısını söyledi. Hoca "her zaman mı?" diye sordu. Bir öğrenci başlangıç düğümünden ulaşılabilen en uzak düğüm sayısını, bir başkası kaynak düğümün aranan düğüme uzaklığını önerdi. Hoca ikisini de kabul etmedi. Burada özel bir arama probleminden değil, DFS ve BFS ile bütün grafın tek tek gezilmesinden söz edildiğini, kaynağın da herhangi bir düğüm seçilebileceğini söyledi.

**Soru:** Arama yaptığımız için aradığımızı bulunca aramayı bırakmayacak mıyız? **Cevap:** En kötü durum senaryosu konuşuluyor. En kötü durumda bütün grafı gezmek gerekir. Zaman ve yer karmaşıklığı bu senaryo üzerinden konuşulur.

Sonunda bir öğrenci eksik parametreyi buldu: komşuluğun nasıl tutulduğu.

> **Hocanın vurgusu:** Bilgilerin hangi yapıda tutulduğu karmaşıklığı doğrudan etkiler. Bu yüzden saklama yapısı da karmaşıklığı belirleyen bir parametredir. Yalnız düğüm ve kenar sayısı yetmez.

### 5.3 Komşuluk Matrisi ve Komşuluk Listesi ile Karmaşıklık

Hoca karmaşıklığı iki durum için ayrı yazdı: komşuluk matrisi ile verimlilik ve komşuluk listesi ile verimlilik.

Önce yer karmaşıklığı konuşuldu. Bir öğrenci matriste $|V|^2$ dedi. Hoca bunun yer karmaşıklığı için doğru olduğunu söyledi: $\Theta(|V|^2)$. Çünkü matris düğümlere göre oluşturulur. Kaç düğüm varsa her birinin diğerleriyle bağlantı olma ihtimali düşünülerek bir matris kurulur. Bir başka öğrenci, komşuluk listesinde yerin düğüm ve kenar sayılarının toplamı kadar olacağını söyledi: $\Theta(|V| + |E|)$.

Ardından hoca zaman karmaşıklığını tahtaya yazdı ve yorumlamalarını istedi. Bir öğrenci şöyle açıkladı:

- **Matris:** Bir düğümün hangi düğümlere bağlı olduğunu bulmak için matrisin o satırını baştan sona gezmek gerekir. Bu her satır için yapıldığından karmaşıklık $\Theta(|V|^2)$ olur.
- **Liste:** Listede yalnız var olan elemanlar gezilir. Her düğümün kenarları ve düğümlerin kendileri toplanır. Karmaşıklık $\Theta(|V| + |E|)$ olur.

**Soru:** Bir düğüm ve kenar sayısı için hangi yöntemin daha avantajlı olduğuna örnek verebilir misiniz? **Cevap:** Bir öğrenci 3 düğümlü bir örnek verdi. Matris kullanılırsa $3 \times 3 = 9$ hücrelik bir matris oluşturmak ve 9 geçiş yapmak gerekir. Graf bir ağaç biçimindeyse, örneğin A düğümünden başlayınca yalnız 2 bağlantı olduğu için 2 adım daha gidilir ve 3 adımda iş biter. Yani komşuluk listesiyle gezinti çok daha kısa sürer.

BFS'in karmaşıklığı sorulduğunda öğrencilerden cevap gelmedi. Hoca cevabı kendisi verdi: BFS'in karmaşıklığı da matriste $\Theta(|V|^2)$, komşuluk listesinde $\Theta(|V| + |E|)$'dir. Hoca, öğrencilerin yanlış da olsa cevap vermeyi denemelerini tercih ettiğini söyledi.

| | DFS | BFS |
|---|---|---|
| Veri yapısı | Yığın | Kuyruk |
| Düğüm sıralaması sayısı | 2 (push ve pop sırası) | 1 |
| Kenar türleri | Ağaç ve geri kenarlar | Ağaç ve çapraz kenarlar |
| Karmaşıklık (komşuluk matrisi) | $\Theta(\|V\|^2)$ | $\Theta(\|V\|^2)$ |
| Karmaşıklık (komşuluk listesi) | $\Theta(\|V\| + \|E\|)$ | $\Theta(\|V\| + \|E\|)$ |

### 5.4 Neden $\Theta$?

**Soru:** Burada $|V|^2$ derken aslında en kötü durumu söylemiyor muyuz? Yine de $\Theta$ ile ifade ediyoruz, değil mi? **Cevap:** Hoca bunun güzel bir soru olduğunu söyledi. Alt sınırın ve üst sınırın birleştiği noktanın $|V|^2$ olması beklenir. Alt sınır, örneğin aranan düğümü ilk gözde bulmak olarak düşünülebilir. Ama burada DFS ile bütün grafı gezme işinden söz ediliyor. Bütün graf gezildiği için en iyi durum da en kötü durum da aynı yere çıkar. Bu yüzden karmaşıklık $\Theta$ ile verilir. Başka bir problem olsaydı, örneğin DFS ile grafın döngülü olup olmadığı araştırılsaydı, gösterim ve en iyi ile en kötü durum ifadeleri değişirdi.

## 6. DFS ve BFS ile Çözülen Problemler

Hoca, DFS ve BFS'in birçok problemi çözdüğünü hatırlatıp birkaç örnek sordu.

**Labirent.** Tek çıkışı olan bir labirentte çıkış noktası aranıyor. Bir öğrenci DFS'i önerdi. Hocanın açıklaması: Gidilebilecek noktaya kadar gidilir. Çıkmaza girince bir adım geri gelinir ve o noktadan gidilebilecek yere kadar yeniden ilerlenir. BFS kullanılsaydı her baktığımız noktada bir adım ilerleyip tekrar geri gelmek gerekirdi. Bu çok uygulanabilir (feasible) değildir.

**GPS.** GPS bir sonraki noktaları sizin adınıza bulur. Bir öğrenci, en kısa yolu bulmak gerektiği için BFS'in daha mantıklı olduğunu söyledi. Hoca onayladı ve GPS'teki mantığın size en yakın noktaları tespit etmek olduğunu ekledi. Bu da en yakın, olası komşu noktalara bakıp komşuları belirlemek demektir.

> **Ders sonrası not:** BFS, yalnız ağırlıksız graflarda en az kenarlı yolu verir. Yol ağları ağırlıklıdır (mesafe, süre gibi). Ağırlıklı grafta en kısa yol için büyük olasılıkla Dijkstra ya da A* gibi algoritmalar kullanılır. Dijkstra derste en kısa yol algoritması olarak zaten anılmıştı.

**Döngü bulma.** Bir öğrenci BFS ile yapılabileceğini, çünkü BFS'in kendi yakınlarından başlayıp etrafında dolaştığı için döngüyü en kısa zamanda bulabileceğini söyledi. Hoca bunu tartışmaya açık buldu. Her ikisiyle de yapılabileceğini, DFS'te alt noktaya erişip geri dönüldüğünde BFS'te de aynı adım sayısıyla erişilebilen örnekler olduğunu, bu yüzden ikisinin de hemen hemen aynı verimlilikle döngü bulacağını söyledi.

**Bağlılık.** Grafın bağlı olup olmadığı da DFS ve BFS ile bulunabilir.

> **Ek bilgi (derste anlatılmadı):** Yönsüz grafta döngü, DFS'te bir geri kenar ya da BFS'te bir çapraz kenar görülmesiyle anlaşılır. Yönlü grafta ise döngü olup olmadığı genelde DFS ile, gezinti sırasında yığında hâlâ bulunan bir düğüme giden kenar (geri kenar) aranarak kontrol edilir.

## 7. Topolojik Sıralamaya Giriş

### 7.1 Problem ve Graf Modeli

Hoca, topolojik sıralamanın (topological sorting) bir adımdan geçmeden diğer adıma ulaşma şansı olmayan problemler için kullanıldığını söyledi. Öğrencilere en tanıdık örnek ders seçimidir. Bir dersi almadan diğer dersi alma şansınız olmayabilir. Buna ön koşul denir. Bir dersin ön koşulu varsa önce o ön koşul alınmalıdır.

**Soru:** Bu durumu hangi tip bir grafla modelleyebiliriz? **Cevap:** Yönlü graf (directed graph).

Hocanın örneği:

- C1 → C3
- C2 → C3
- C3 → C4
- C3 → C5
- C4 → C5

**Soru:** Bu graf döngülü mü, döngüsüz mü? **Cevap:** Döngüsüz (acyclic). Hoca nedenini açıkladı: C3, C4, C5 arasında bir döngü varmış gibi görünse de döngüyü bozan şey okların yönüdür. C3'ten C5'e ok vardır, ama C5'ten C3'e ok yoktur. C3'ten başlayıp C4 ve C5 üzerinden yeniden C3'e dönülemez.

### 7.2 Ön Koşulların Okunması

**Soru:** Hangi ders hangisinin ön koşuludur? **Cevap:** Bir öğrenci şöyle cevap verdi: C1 ve C2, C3'ün ön koşuludur. C3, C4'ün ön koşuludur. C3 ve C4 birlikte C5'in ön koşuludur. Hoca, bir derse giriş yapan bir ok varsa o dersin ön koşulu olduğunu söyledi. C3'ün ön koşulları C1 ve C2'dir. C1 ve C2'ye ok girmez, bu yüzden ön koşulsuz derslerdir.

Ardından bir öğrenci itiraz etti: C5'e erişmek için C4'e ihtiyaç yoktur, C3'ten de doğrudan erişilebilir. Aynı şekilde C3'e erişmek için C2'ye ihtiyaç yoktur, C1 de yeterli olabilir. Hoca bu düzeltmeyi kabul etti ve "ve" ifadesini kullanmanın hata olduğunu, C3'e ulaşmak için C1 veya C2'nin alınması gerektiğini söyledi. İkisinin aynı anda gerekmesi için grafın farklı çizilmesi gerektiğini söyleyerek bir örnek çizdi.

> **Ders sonrası not:** Hoca, C3'ü almak için C1 veya C2'den birinin yeterli olduğunu kabul etti. Doğrusu: Öncelik grafında $u \to v$ kenarı "$u$, $v$'den önce gelmeli" anlamına gelir. C3'e iki kenar giriyorsa C1 ve C2'nin ikisi de C3'ten önce gelmelidir. Yani ön koşullar VE ile bağlıdır. İlk öğrencinin cevabı doğruydu. Hocanın birkaç dakika sonra verdiği iki geçerli sıra (C1, C2, … ve C2, C1, …) ve "C3 ile başlayamayız" demesi de bu anlama uyar. Python ile bütün sıralar denendi: VE anlamında geçerli sıra tam olarak bu ikisidir.

### 7.3 DFS ile Gezinti

**Soru:** Burada DFS mi kullanılabilir, BFS mi? **Cevap:** Bir öğrenci BFS'i önerdi, çünkü hangi düğümden gelindiği önemlidir, en uzun yol önemli değildir. Hoca bunun bir bakış açısı olduğunu, ama DFS üzerinden ilerlemenin daha uygun olabileceğini düşündüğünü söyledi. Önce DFS ile çözüp sonra BFS ile olup olmayacağına bakılacağını belirtti.

DFS ile, eşitlikte küçük numaralı düğüm seçilerek ilerlenir:

- C1 ile başlanır. C1 yığına eklenir.
- C1'den C3'e gidilir. C3 eklenir.
- C3'ten iki seçenek vardır: C4 ve C5. Küçük olan C4 seçilir. C4 eklenir.
- C4'ten tek seçenek C5'tir. C5 eklenir.
- C5'in komşusu yoktur. Yığından çıkarma başlar. C5 çıkarılır.
- C4'ün gidebileceği başka komşu yoktur. C4 çıkarılır.
- C3'ün de komşusu kalmamıştır. C2, C3'e tersten bağlıdır, yani C3'ten C2'ye ok yoktur. C3 çıkarılır.
- C1'in de başka bağlantısı yoktur. C1 çıkarılır.

Beş düğümün dördüne erişilmiş, ama yığın boşalmıştır. DFS ile gezinti bitmemiştir. Dıştaki `for` döngüsü ziyaret edilmemiş C2'yi bulur ve yığına koyar. C2'nin gidecek komşusu kalmadığı için C2 hemen çıkarılır.

Python ile doğrulanan sıralar:

| Sıra | Düğümler |
|---|---|
| Yığına ekleme (push) sırası | C1, C3, C4, C5, C2 |
| Çıkarma (popping order) sırası | C5, C4, C3, C1, C2 |

### 7.4 Dersler Hangi Sırayla Alınabilir?

**Soru:** Dersler hangi sırayla alınabilir? **Cevap:** Hoca C2, C1, C3, C4, C5 sırasının seçeneklerden biri olduğunu söyledi. Bir öğrenci C1, C2, C3, C4, C5 sırasını ekledi. Hoca başka seçenek olmadığını söyledi: C3, C4 ya da C5 ile başlanamaz. Yani iki seçenek vardır.

Hoca, DFS ile gezerken her adımda nereye gidileceğinin ön koşullara (prerequisite) dikkat edilerek bulunduğunu ve sonunda en son alınacak derse varıldığını söyledi. Ardından dersin son sorusunu sordu.

**Soru:** DFS'te adım adım ilerlerken alınacak ders sırasını ekrana yazdırmak istiyorum. Bunu push sırasında mı, pop sırasında mı yapmalıyım? **Cevap:** Bir öğrenci push sırasını önerdi: Yığına attıkça yazdırırsak doğru sıra olur. Bir başka öğrenci daha önce pop sırasının tersini söylemişti. Hoca cevabı vermedi. Konuyu aradan sonraki uygulama saatinde toparlayacağını ve problemin başka bir bakış açısıyla çözülüp çözülemeyeceğini soracağını söyledi.

> **Ders sonrası not:** Push sırası (C1, C3, C4, C5, C2) geçerli bir topolojik sıra değildir, çünkü C2, C3'ten sonra gelir. Pop sırasının tersi (C2, C1, C3, C4, C5) geçerlidir ve hocanın verdiği iki seçenekten biridir. Kısa nota göre cevap ders 22'de verildi: çıkarma sırasının tersi.

## Örnek Sorular

Bu derste ekran kaydı ya da kare yoktur. Sorular, derste sözlü sorulan sorulardan alındı.

**1. (derste sözlü, 00:28:41)** LCS'nin harflerini bulmak için $B[i, j]$ matrisine nereden itibaren bakılır?

> **Çözüm:** Sağ alt köşeden ($i = m$, $j = n$). Geri izleme döngüsü $i \neq 0$ ve $j \neq 0$ olduğu sürece çalışır. add görülünce $x_i$ LCS'nin önüne eklenir ve iki indis birlikte azalır. skip x görülünce $i$, skip y görülünce $j$ azalır. BACDB ve BDCB için, eşitlikte skip x seçilirse sonuç BCB olur.

**2. (derste sözlü, 00:34:48)** Bir grafın ağaç olma koşulu nedir?

> **Çözüm:** Derste cevap "döngü oluşmaması" olarak verildi. Tam koşul: graf bağlı ve döngüsüz olmalıdır (bkz. 2.2'deki ders sonrası not).

**3. (derste sözlü, 00:38:28)** Komşuluk matrisi mi, komşuluk listesi mi kullanılacağına neye göre karar verilir?

> **Çözüm:** Kenar ve düğüm sayısının birlikte durumuna göre. Yoğun grafta (kenar sayısı $|V|^2$ mertebesinde) komşuluk matrisi, seyrek grafta komşuluk listesi tercih edilir.

**4. (derste sözlü, 00:41:48)** Ağırlıklı grafta iki düğüm arasında bağlantı yoksa matriste ne yazılır? Listede ne saklanır?

> **Çözüm:** Matriste $\infty$ (çok büyük bir sayı) yazılır. Listede her komşu için komşu düğüm ve kenarın ağırlığı birlikte saklanır.

**5. (derste sözlü, 00:44:53)** A-B, B-C, B-D, B-E, D-E ve F-G kenarlı grafta kaç bağlı bileşen vardır? DFS ile bu sayı nasıl bulunur?

> **Çözüm:** İki bileşen vardır: ABCDE ve FG. DFS'in dıştaki döngüsünde $dfs(v)$ her çağrılıp döndüğünde bir bileşen bitmiş olur. Sayaç oraya konur. Bu grafta $dfs$ A ve F için çağrılır, sonuç 2'dir. Push sırası A, B, C, D, E, F, G; pop sırası C, E, D, B, A, G, F'dir (Python ile doğrulandı).

**6. (derste sözlü, 00:54:08)** DFS'te yığından pop işlemi ne zaman yapılır?

> **Çözüm:** Ulaşılan düğümün ziyaret edilmemiş komşusu kalmadığında. Pop yapılınca bir önceki düğüme dönülür ve onun komşularına bakılır.

**7. (derste sözlü, 01:13:08)** BFS'te özyinelemeli mi, yinelemeli mi bir yaklaşım kullanılır?

> **Çözüm:** Yinelemeli. Derste "özyinelemeli" dendi, ama yazılan $bfs(v)$ kuyruk ve `while` döngüsüyle çalışır, kendini çağırmaz (bkz. 4.1'deki ders sonrası not).

**8. (derste sözlü, 01:24:19)** DFS'te ve BFS'te kaç düğüm sıralaması vardır?

> **Çözüm:** DFS'te 2 (push ve pop sırası), BFS'te 1.

**9. (derste sözlü, 01:25:52)** DFS ve BFS'in karmaşıklığını belirleyen parametreler nelerdir? Komşuluk matrisi ve komşuluk listesi için karmaşıklık nedir?

> **Çözüm:** Düğüm sayısı, kenar sayısı ve grafın saklandığı yapı. İki algoritma için de matriste $\Theta(|V|^2)$, listede $\Theta(|V| + |E|)$. Bütün graf gezildiği için en iyi ve en kötü durum aynıdır, bu yüzden $\Theta$ kullanılır.

**10. (derste sözlü, 01:36:33)** Tek çıkışlı bir labirentte çıkışı bulmak için DFS mi, BFS mi tercih edilir?

> **Çözüm:** DFS. Gidilebilecek yere kadar ilerlenir, çıkmaza girince bir adım geri dönülür.

**11. (derste sözlü, 01:41:48)** C1 → C3, C2 → C3, C3 → C4, C3 → C5, C4 → C5 grafı döngülü müdür? Dersler hangi sıralarla alınabilir?

> **Çözüm:** Graf döngüsüzdür, çünkü C5'ten C3'e ok yoktur. Geçerli sıralar C1, C2, C3, C4, C5 ve C2, C1, C3, C4, C5'tir (Python ile bütün permütasyonlar denendi).

**12. (derste sözlü, 01:52:20)** DFS ile gezerken topolojik sırayı yazdırmak için push sırası mı, pop sırası mı kullanılır?

> **Çözüm (derste verilmedi):** Pop sırasının tersi. Örnekte pop sırası C5, C4, C3, C1, C2'dir. Tersi C2, C1, C3, C4, C5'tir ve geçerlidir. Push sırası C1, C3, C4, C5, C2 geçersizdir, çünkü C2, C3'ten sonra gelir (Python ile doğrulandı).

## Dersin Toparlaması

Ders, dinamik programlamanın son parçasıyla başladı. LCS'nin uzunluğunu veren $C$ matrisinin yanına bir $B$ yön matrisi eklendi. Her hücrede ekleme, x'te atlama ya da y'de atlama işareti saklandı. Sağ alt köşeden başlayan geri izleme koduyla örüntünün kendisi bulundu. Eşitlik durumunda seçilen yön, birden fazla LCS olduğunda hangisinin bulunacağını belirler.

Ardından graflara geçildi. Kullanım alanları, yönlü ve yönsüz graf, komşuluk matrisi ve listesi, yoğun ve seyrek graf, tam ve ağırlıklı graf, döngüsüzlük, bağlılık, bağlı bileşen ve orman kavramları hatırlatıldı. DFS'in yığınla ve özyinelemeyle, BFS'in kuyrukla çalıştığı sözde kodlar ve örneklerle gösterildi. İki algoritmanın da bütün grafı gezdiği için karmaşıklığı $\Theta$ ile verildi: matriste $\Theta(|V|^2)$, listede $\Theta(|V| + |E|)$. Labirent, en yakın noktalar, döngü bulma ve bağlılık gibi kullanımlar konuşuldu. Ders, ön koşul grafı örneğiyle topolojik sıralamaya girişle bitti. DFS sırasında hangi bilgiyle sıralamanın yazdırılacağı sorusu açık bırakıldı. Konu, aradan sonraki uygulama saatinde tamamlanacaktı.

**Not:** İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.